

Scheduling Jobs on Parallel Machines with Sequence-Dependent Setup Times, Precedence Constraints, and Ready Times Using Variable Neighborhood Search

Rene Drießel, Lars Moench

Department of Mathematics and Computer Science, University of Hagen, Hagen, Germany
({Rene.Driessel|Lars.Moench}@fernuni-hagen.de)

ABSTRACT

In this paper, we discuss a scheduling problem for jobs on identical parallel machines. Ready times of the jobs, precedence constraints, and sequence-dependent setup times are considered. We are interested in minimizing the performance measure total weighted tardiness. Scheduling problems of this type are quite important in semiconductor manufacturing. We suggest a variable neighborhood search (VNS) scheme for this problem. Based on computational experiments with stochastically generated test instances we are able to show that the VNS approach clearly outperforms heuristics based on the Apparent Tardiness Cost with Setups and Ready Times (ATCSR) dispatching rule in many situations.

Keywords: Scheduling, Parallel Machines, Total Weighted Tardiness, Variable Neighborhood Search

1. INTRODUCTION

Advanced scheduling techniques are quite important in complex manufacturing systems like, for example, in semiconductor wafer fabrication facilities (wafer fabs). Here, we have to deal with parallel machines, sequence-dependent setup times, and reentrant flows of the jobs. Automated material handling systems (AMHS) are very common in this type of manufacturing systems.

Some recent efforts are made to apply the shifting bottleneck heuristic (SBH) to schedule jobs in wafer fabs (cf., for example, [8] for a distributed variant of the SBH). The SBH for wafer fabs decomposes the overall scheduling problem into a sequence of scheduling problems for parallel machines. We call these smaller scheduling problems sub problems. A scheduling graph is used to connect the solutions of the sub problems to compute a solution of the entire scheduling problem.

Only some preliminary results are available for applying the SBH to wafer fabs with AMHS (cf. [2] for a description of a modified SBH with these characteristics). One of the main differences to the SBH for wafer fabs without AMHS is a new sub problem that is related to transportation tasks. Based on the analogy of traveling salesman like problems and scheduling problems with sequence-dependent setup times, it turns out that the sub problem can be transformed into a scheduling problem for jobs on identical parallel machines with sequence-dependent setup times. Therefore, we are interested in coming up with efficient computational schemes for solving these problems. We are especially interested in solution techniques that derive a solution with a small amount

of time because the SBH requires the frequent solution of such problems.

In this paper, we are mainly interested in VNS schemes because it is well known from the literature that VNS is able to provide high quality solutions for a large class of combinatorial optimization problems very quickly.

The paper is organized as follows. In Section 2, we present a problem description and discuss related literature. We introduce the suggested VNS scheme in Section 3. The design of experiments and the results of computational experiments are discussed in Section 4.

2. PROBLEM AND PREVIOUS WORK

2.1 Problem Setting

The scheduling problem to be researched can be presented using the $\alpha|\beta|\gamma$ classification scheme for scheduling problems. Here, the α field is used to describe the machine environment, the β field is used to describe process conditions like ready times or sequence dependent-setup times. Finally, the γ field is used to represent the objective function. Using this scheme, the scheduling problem can be represented in the form

$$Pm|r_j, s_{kj}, prec|\sum w_j T_j, \quad (1)$$

where we denote by Pm identical parallel machines, r_j is used to denote the ready time of job j , s_{kj} denotes sequence-dependent setup times, and the notation $prec$ refers to precedence constraints. Note that the precedence constraints are a result of the SBH where precedence constraints have to be fulfilled to avoid cycles in the scheduling graph.

The performance measure of interest is the total weighted tardiness (TWT) of the jobs. The TWT quantity is the sum of the individual tardiness $T_j := \max(C_j - d_j, 0)$ of job j , multiplied with a job specific weight w_j . The weights are used to model different importance of the customers. Minimizing TWT will ensure a good on-time delivery performance in complex manufacturing systems. It is quite important in due date oriented manufacturing environments.

The scheduling problem (1) is NP hard because the simple single scheduling problem $1||TWT$ that is included in problem (1) as a special case is also NP hard. Hence, we have to look for efficient heuristics.

2.2 Related Literature

Solving scheduling problems with sequence-dependent setup times is a very active research area (cf. [1] for a recent survey). Lee and Pinedo [4] suggested a modification of the ATC rule that takes sequence-dependent setup times into account. The resulting dispatching rule is called ATCS. But it does not include ready times of the jobs.

Extensions of ATCS for situations that include ready times of the jobs are suggested by Pfund *et al.* [10]. They also present results of experiments related to an efficient parameterization of the ATCS rule including a grid search and a regression analysis approach.

Ovacik and Uzsoy [9] suggested a time based decomposition approach for $Pm|r_j, s_{kj}|L_{max}$ where we denote by L_{max} the maximum lateness of the jobs. The authors used dispatching rules to select jobs within a certain time window. Then a branch and bound scheme or local-search techniques are applied to schedule these jobs.

Liao and Juan [5] suggested an ant colony system (ACS) for solving $1|s_{kj}|TWT$. Within ACS, they used the ATCS dispatching rule to compute heuristic information for the job sequences. Rocha *et al.* [11] suggested a VNS and a GRASP approach for the parallel machine scheduling problem $Pm|s_{kj}|C_{max} + TWT$ where we denote by C_{max} the makespan of the schedule. The VNS approach outperforms the GRASP scheme, especially for large-size test instances.

Wang and Tang [12] described a population based VNS scheme for $1||TWT$. Several meta-heuristics, including genetic algorithms, tabu-search, and simulated annealing are compared by Lin and Ying [6] for the same scheduling problem.

To our best knowledge, the problem researched in this paper is not considered by other authors.

3. VARIABLE NEIGHBORHOOD SEARCH

3.1 Principles of VNS

Variable neighborhood search is a relatively new local-search based metaheuristic (cf. [3,7] for a detailed description). The main idea is to enrich a simple local search method in order to enable it escaping from local optima. This is done by restarting the local search from a randomly chosen neighbor of the incumbent solution. This restarting step, called shaking, is performed using different neighborhoods of increasing sizes.

There are many different variations of that VNS idea, such as variable neighborhood descent (VND) or reduced variable neighborhood search (RVNS).

The performance of a VNS scheme is mainly influenced by the following factors

- the quality of the used initial solution,
- the selected neighborhoods,
- the used local-search techniques,
- the order how the neighborhoods are applied.

3.2 Used Neighborhood Structures

In this paper, a solution of problem (1) is defined as a job sequence I_i for each machine i . Let S be the set of all possible feasible solutions. For a solution $x = \{I_1, I_2, \dots, I_m\} \in S$, several neighborhood structures $N_k(x) \subseteq S$ can be defined. The definition of the neighborhoods is done by describing how to modify the current (feasible) solution to generate a new (feasible) solution. In this paper, we define the following four different basic neighborhoods. They are also applied in the following order:

1. Job transfers between machines: one job on one machine is chosen and all possible job transfers to all machines are considered.
2. Job swaps between machines: one job on one machine is chosen and all possible job swaps are considered.
3. Job transfer on one machine: one job on one machine is chosen and all possible job transfers on this machine are considered.
4. Job swaps on one machine: one job on one machine is chosen and all possible job swaps on this machine are considered.

Because we have to consider precedence constraints between the jobs it has to be ensured that only those transfers and swaps are allowed that do not lead to an infeasible solution.

If a job transfer is done it has to be ensured that the job is not transferred before a direct or indirect predecessor or after a direct or indirect successor. Therefore, for each job j in a given solution x a maximum lower and minimum upper position on each machine i is calculated. Between these positions the movement of the job does not make the schedule infeasible.

Because a swap can be defined as two job movements it is sufficient to make sure that the position of

the second job is a valid move position for the first and vice versa.

Furthermore, each of the four basic neighborhoods is respectively enlarged by iterating the set of defining moves two and three times to form larger neighborhoods. Totally, we use 12 different neighborhoods (four basic and eight derived neighborhoods).

3.3 Initial Solution

Any method that is able to generate a feasible solution can be used to obtain an initial solution. However it is obvious that a good initial solution can reduce considerably the computation time of the VNS algorithm (cf. [11] for similar observations for a related scheduling problem).

In this paper, we use therefore the Apparent Tardiness Cost with Setups and Ready times (ATCSR) dispatching rule. It is well known that this dispatching rule outperforms many other dispatching rules (cf. [10] a detailed study for the performance of ATCSR). The ATCSR index for job j is given by

$$I_j(t) = \frac{w_j}{p_j} e^{\left[\frac{\left(d_j - p_j - t + (r_j - t)^+ \right)^+}{k_1 \bar{p}} + \frac{s_{ij}}{k_2 \bar{s}} + \frac{(r_j - t)^+}{k_3 \bar{p}} \right]}. \quad (2)$$

Here, the notation $x^+ := \max(x, 0)$ is used for abbreviation. The processing time for job j is denoted with by p_j , d_j is used for the due date, r_j for the release date, and w_j for the weight of job j . The current time is denoted by t . The quantity s_{ij} is the setup time needed for the setup from the current job i to job j . The average processing time of the remaining jobs available is denoted by $\bar{p}$. The notation $\bar{s}$ is used for the average setup time.

The ATCSR index is a composite priority index to estimate the urgency of selecting a job. The job with the highest index is chosen next within a list based scheduling approach. The slack term is scaled with the parameter k_1 , the term related to setup with the parameter k_2 , and the release term with k_3 . The selection of the three parameters is critical for the solution quality obtained by ATCSR. In this paper, we use a grid search approach similar to [10] to generate a set of schedules. The schedule that leads to the lowest TWT value is used as the initial solution for our VNS algorithm.

The grid used follows the suggested one in [10]. There are 22 values for k_1 , 11 values for k_2 , and 13 values for k_3 . All triples (k_1, k_2, k_3) obtained in such a manner are used within index (2).

Therefore, we end up with 3146 different schedules. The used grid points for k_1, k_2 , and k_3 are

- $k_1 \in \{0.2, 0.6, 0.8, 1, 1.2, 1.4, 1.6, 1.8, 2, 2.4, 2.8, 3.2, 3.6, 4.0, 4.4, 4.8, 5.2, 5.6, 6.0, 6.4, 6.8, 7.2\}$,
- $k_2 \in \{0.1, 0.3, 0.5, 0.7, 0.9, 1.1, 1.3, 1.5, 1.7, 1.9, 2.1\}$,
- $k_3 \in \{0.001, 0.0025, 0.004, 0.005, 0.025, 0.04, 0.05, 0.25, 0.4, 0.6, 0.8, 1, 1.2\}$.

This grid search is time consuming. Therefore, we also test a smaller grid with seven values for k_1 , four values for k_2 , and five values for k_3 . We apply the following grid points

- $k_1 \in \{0.2, 1.0, 1.6, 2.4, 3.6, 4.8, 6.0\}$,
- $k_2 \in \{0.1, 0.7, 1.3, 1.9\}$,
- $k_3 \in \{0.001, 0.005, 0.05, 0.6, 1.2\}$.

In this case, we have to consider only 140 different schedules.

3.4 Proposed VNS Scheme

The used VNS scheme is illustrated in Figure 1. We terminate the VNS search after a specific amount of time. In the local-search, we look for the best neighbor within the current neighborhood.

```

Find a solution  $s$ ;
 $k \leftarrow 1$ ;
while maximum runtime is not reached do
    Shaking: select a random solution  $x \in N_k(s)$ ;
    Exploration of neighborhood: find the best
     $x' \in N_k(x)$  by consideration all points of
     $N_k(x)$ 
    Move or not:
    if solution  $x'$  is better  $s$  then
         $s \leftarrow x'$ ;
         $k \leftarrow 1$ ;
    else
         $k \leftarrow (k \bmod 12) + 1$ ;
end

```

Figure 1: Proposed VNS algorithm

When a new neighborhood is chosen in the shaking procedure a random solution from the neighborhood is selected. The following procedures are used for the four basic neighborhoods:

1. Job transfers between machines:
 - a. Choose randomly two machines i_1 and i_2 .
 - b. Choose randomly a job j_1 on machine i_1 and a valid position on i_2 (the valid position has to be between the maximum upper and minimum lower position on i_2).
 - c. Transfer the job.
2. Job swaps between machines:
 - a. Choose randomly two machines i_1 and i_2 .
 - b. Choose randomly a job j_1 on machine i_1 and a valid swap job j_2 on machine i_2 .
 - c. Do the swap.

3. Job transfer on one machine:
 - a. Choose randomly a machine i .
 - b. Choose randomly a job j_i on machine i and a valid position on the same machine.
 - c. Transfer the job
4. Job swaps on one machine:
 - a. Choose randomly a machine i .
 - b. Choose randomly two jobs j_i and j_j on machine i .
 - c. Do the swap.

These procedures are repeated two or three times for the one after another for the suggested neighborhoods.

The used objective function within the VNS approach is TWT. We use C_{max} as tie breaker when the TWT values of two solutions are equal.

4. COMPUTATIONAL EXPERIMENTS

4.1 Design of Experiments

To evaluate the performance of our VNS approach, we solve instances of problem (1) with the following solution approaches and compare the results:

- FIFO,
- ATCSR with large grid size (ATCSR-LG),
- ATCSR-with small grid size (ATCSR-SG),
- Short running VNS (VNS-S, 1 minute),
- Middle running VNS (VNS-M, 2.5 minutes),
- Long running VNS (VNS-L, 5 minutes).

Each of the three different VNS schemes starts with the initial solution from ATCSR-LG (VNS-LG-S, VNS-LG-M, VNS-LG-L) and then with the initial solution from ATCSR-SG (VNS-SG-S, VNS-SG-M, VNS-SG-L) to compare the influence of the grid size on the overall performance.

The following seven factors are assumed to characterize a problem instance. Note that the first six factors are from the design used in [10]. The method for generating the problem instances has been adapted from [4,10] and is described below. We use five machines, i.e., $m = 5$. The number of jobs is determined by the job machine factor μ . It is the average number of jobs processed per machine. It holds $n = m \cdot \mu$. In our experiments, we use the values $\mu = \{11, 19, 27\}$. The processing times p_j are uniformly distributed over the interval $[50, 150]$. The weights w_j are uniformly distributed integers over the interval $[1, 10]$. The mean setup time $\bar{s}$ is calculated using the setup servity factor η that characterizes the effect of setup times on the schedule. It holds $\bar{s} = \eta \cdot \bar{p}$. We use $\eta \in \{0.020, 1.010, 2.000\}$. The makespan is estimated by $C_{max} = (\beta \bar{s} + \bar{p})\mu$, where β is a coefficient accounting the increase in makespan due to setup times. It is given by $\beta = 0.4 + 10/\mu^2 - \eta/7$ (cf. [4,10]). The mean due date

$\bar{d}$ is calculated using the due date tightness factor $\tau \in \{0.300, 0.600, 0.900\}$ that describes how close is the average due date is to C_{max} . We use the relation $\bar{d} = C_{max}(1 - \tau)$. We use a due date range factor $R \in \{0.250, 0.630, 1.000\}$ to generate due dates. The due dates are uniformly distributed over the interval $[(1 - R)\bar{d}, \bar{d}]$ with probability τ and over the interval $[\bar{d}, \bar{d} + (C_{max} - \bar{d})R]$ with probability $(1 - \tau)$. The job available factor is used to assign ready times of 0 with probability $J_a = \{0.200, 0.500, 0.800\}$ to the jobs. The ready times are uniformly distributed over the interval $[\max(d_j - r_\tau \cdot p_j, 0), d_j]$ with probability $(1 - J_a)$, where r_τ is the ready time tightness factor. We use $r_\tau \in \{1.000, 5.500, 10.000\}$. Finally the precedence relations are inserted using the precedence factor $\pi \in \{0.250, 0.500, 0.750\}$. If a chosen random number from $[0, 1]$ is larger than π no predecessors are added to the job. If the random number is lower than π the amount of predecessors is chosen uniformly distributed over the interval $[0, \min(3.0, \pi \cdot n_{cur})]$, where n_{cur} is the number of already generated jobs. The determined amount of jobs is now randomly picked from the set of already generated jobs. Precedence constraints between the selected jobs and the current job are introduced. Furthermore, the release and due date of the current job is adjusted accordingly.

Table 1 summarizes the factors used for the experimentation. Each combination of the levels is used within the experiments. Thus we have totally 2187 different problem instances. Therefore, we have to carry out 19683 experiments for the nine different algorithms.

Table 1: Values of factors for experimentation

Factor/Level	Low	Mid	High
Job machine factor	11.000	19.000	27.000
Setup servity factor	0.020	1.010	2.000
Due date tightness factor	0.300	0.600	0.900
Due date range factor	0.250	0.630	1.000
Job availability factor	0.200	0.500	0.800
Ready time factor	1.000	5.500	10.000
Precedence factor	0.250	0.500	0.750

4.2 Results

Table 2 shows the results of the experiments for ATCSR-LG, while Table 3 shows the same results for ATCSR-SG. The results are presented as average values relative to the TWT values obtained by a FIFO dispatching rule. These values are aggregated for each of the levels. FIFO is used for comparison to show the differences in the obtained solution quality for ATCSR type rules with respect to the chosen granularity of the grid.

Table 4 shows the average results, the standard deviation, and the best and worst case performance of

the tested algorithms also relative to FIFO. Finally, the average improvements of the initial solutions for our VNS approach are shown in Table 5 and Table 6.

Table 2: Results relative to FIFO for VNS with large grid search

Aggregation	ATCSR	VNS-S	VNS-M	VNS-L
$\mu = 11$	0.4822	0.4730	0.4444	0.4279
$\mu = 19$	0.4272	0.4241	0.4155	0.4055
$\mu = 27$	0.4780	0.4758	0.4739	0.4712
$\eta = 0.020$	0.7684	0.7565	0.7260	0.7045
$\eta = 1.010$	0.3141	0.3126	0.3076	0.3030
$\eta = 2.000$	0.2462	0.2455	0.2413	0.2379
$\tau = 0.3000$	0.3564	0.3504	0.3307	0.3171
$\tau = 0.6000$	0.4508	0.4457	0.4331	0.4229
$\tau = 0.9000$	0.5734	0.5693	0.5592	0.5517
$R = 0.250$	0.4739	0.4692	0.4556	0.4457
$R = 0.630$	0.4661	0.4611	0.4476	0.4378
$R = 1.000$	0.4427	0.4371	0.4219	0.4104
$J_a = 0.200$	0.4576	0.4525	0.4374	0.4278
$J_a = 0.500$	0.4642	0.4590	0.4454	0.4348
$J_a = 0.800$	0.4614	0.4564	0.4429	0.4319
$r_\tau = 1.000$	0.4580	0.4538	0.4407	0.4305
$r_\tau = 5.500$	0.4643	0.4585	0.4439	0.4339
$r_\tau = 10.000$	0.4607	0.4556	0.4411	0.4300
$\pi = 0.250$	0.3379	0.3353	0.3293	0.3245
$\pi = 0.500$	0.4497	0.4445	0.4313	0.4211
$\pi = 0.750$	0.5953	0.5879	0.5649	0.5486

4.3 Discussion

As shown in Table 6, the short running VNS improves the initial solution about 1-2% in average. However, for a long running VNS the average improvement is about 6-7%. An improvement up to 50% is possible (see Table 6). It seems to be interesting that the usage of the VNS algorithms also reduces the standard deviation of the results (see Table 4).

Some problem instance factors have influence on the results of our approach. These factors are namely the precedence factor π , the due date tightness factor τ , the setup servity factor η , and the job machine factor μ . All these factors determine how hard the test instance is with respect to the TWT objective. If the problem becomes more complicated our approach provides better results (see Table 2 and 3).

As shown in Table 5, VNS is able to compensate some drawbacks from the ATCSR heuristic. The ATCSR rule gives rather worse results when using wide due dates or low load on the machines because the slack term becomes in such situations often negative. This leads to a indifference between these jobs. The reason why our VNS approach is able to improve the results lies mainly in the used objective function which considers C_{max} as a tie breaker when the TWT values of different solutions are equal.

Table 3: Results relative to FIFO for VNS with small grid search

Aggregation	ATCSR	VNS-S	VNS-M	VNS-L
$\mu = 11$	0.5011	0.4843	0.4554	0.4365
$\mu = 19$	0.4418	0.4381	0.4273	0.4178
$\mu = 27$	0.4950	0.4929	0.4896	0.4857
$\eta = 0.020$	0.7948	0.7765	0.7440	0.7221
$\eta = 1.010$	0.3270	0.3244	0.3182	0.3121
$\eta = 2.000$	0.2554	0.2538	0.2492	0.2448
$\tau = 0.3000$	0.3811	0.3699	0.3488	0.3342
$\tau = 0.6000$	0.4683	0.4612	0.4471	0.4358
$\tau = 0.9000$	0.5818	0.5759	0.5649	0.5565
$R = 0.250$	0.4890	0.4811	0.4666	0.4564
$R = 0.630$	0.4828	0.4749	0.4613	0.4504
$R = 1.000$	0.4613	0.4528	0.4348	0.4218
$J_a = 0.200$	0.4746	0.4674	0.4520	0.4405
$J_a = 0.500$	0.4804	0.4718	0.4560	0.4443
$J_a = 0.800$	0.4786	0.4702	0.4553	0.4443
$r_\tau = 1.000$	0.4752	0.4676	0.4528	0.4419
$r_\tau = 5.500$	0.4814	0.4730	0.4584	0.4459
$r_\tau = 10.000$	0.4769	0.4687	0.4522	0.4414
$\pi = 0.250$	0.3461	0.3420	0.3358	0.3307
$\pi = 0.500$	0.4649	0.4563	0.4423	0.4312
$\pi = 0.750$	0.6223	0.6109	0.5850	0.5670

Furthermore, because of the direct consideration of the precedence constraints our VNS approach leads to better results as ATCSR when more precedence constraints have to be taken into account. Considering the additional runtime which is given for the VNS algorithms it seems that the additional computation time is worth, especially in situations where ATCSR has some limitations. We also test for some instances longer runtimes for the VNS algorithms but we obtain only small improvement rates in these cases.

Table 4: Results for different algorithms

Algorithm	Average	Standard Deviation	Min	Max
ARCSR-LG	0.4610	0.2939	0.0392	1.5572
VNS-LG-S	0.4560	0.2887	0.0392	1.5572
VNS-LG-M	0.4419	0.2758	0.0392	1.5513
VNS-LG-L	0.4315	0.2674	0.0392	1.5513
ARCSR-SG	0.4778	0.3040	0.0432	1.7818
VNS-SG-S	0.4698	0.2963	0.0432	1.7818
VNS-SG-M	0.4544	0.2824	0.0432	1.7638
VNS-SG-L	0.4431	0.2740	0.0432	1.5392

5. CONCLUSIONS AND FUTURE RESEARCH

In this paper, we discuss a scheduling problem for parallel machines. The jobs have ready times, sequence-dependent setup times, and precedence constraints. Problems of this type arise as sub problem

solution procedures of a modified SBH for complex job shops with automated material handling.

Table 5: VNS improvements over ATCSR-LG

Aggregation	VNS-S	VNS-M	VNS-L
$\mu = 11$	1.91%	7.84%	11.26%
$\mu = 19$	0.73%	2.74%	5.08%
$\mu = 27$	0.46%	0.86%	1.42%
$\eta = 0.020$	1.55%	5.52%	8.32%
$\eta = 1.010$	0.48%	2.07%	3.53%
$\eta = 2.000$	0.28%	1.99%	3.37%
$\tau = 0.3000$	1.68%	7.21%	11.03%
$\tau = 0.6000$	1.13%	3.93%	6.19%
$\tau = 0.9000$	0.72%	2.48%	3.78%
$R = 0.250$	0.99%	3.86%	5.95%
$R = 0.630$	1.07%	3.97%	6.07%
$R = 1.000$	1.26%	4.70%	7.30%
$J_a = 0.200$	1.11%	4.41%	6.51%
$J_a = 0.500$	1.12%	4.05%	6.33%
$J_a = 0.800$	1.08%	4.01%	6.39%
$r_\tau = 1.000$	0.92%	3.78%	6.00%
$r_\tau = 5.500$	1.25%	4.39%	6.55%
$r_\tau = 10.000$	1.11%	4.25%	6.66%
$\pi = 0.250$	0.77%	2.55%	3.97%
$\pi = 0.500$	1.16%	4.09%	6.36%
$\pi = 0.750$	1.24%	5.11%	7.84%

We suggest a VNS approach that is able to outperform schedules that are obtained by a list based scheduling approach using the ATCSR dispatching rule.

There are several directions for future research. First of all, we are interested in improving the quality of the VNS scheme by using other type of neighborhoods. Instead of insert or exchange only single jobs, we expect a better solution quality by moving entire blocks of consecutive jobs because we do not destroy partial sequences that lead to small TWT values. We also believe that rolling horizon approaches as suggested by Ovacik and Uzsoy [9] for a similar scheduling problem can be combined easily with our VNS scheme.

Table 6: Improvements of the initial solution

Algorithm	Average	Standard Deviation	Max
VNS-LG-S	0.78%	2.01%	30.02%
VNS-LG-M	3.11%	5.13%	39.28%
VNS-LG-L	5.00%	6.76%	49.14%
VNS-SG-S	1.25%	2.80%	31.49%
VNS-SG-M	3.70%	5.73%	44.88%
VNS-SG-L	5.74%	7.49%	56.34%

Finally, we are interested in using our VNS scheme as a sub problem solution procedure within a SBH scheme for complex manufacturing systems with automated material handling. We expect an improved solution quality over pure dispatching based sub

problem solution procedures. However, carry out all the necessary details is part of future

REFERENCES

- [1] Allahverdi, A., C. T. Ng, C.T., T. C. E. Cheng, M. Y. Kovalyov, "A survey of scheduling problems with setup times or costs", *European Journal of Operational Research*, 187, pp 985-1032, 2008
- [2] Driessel, R. and L. Mönch, "An integrated scheduling and automated material handling approach for complex manufacturing systems", *Proceedings of the 2008 IEEE International Conference on Industrial Engineering and Engineering Management*, pp 826-830, 2008
- [3] Hansen, P., N. Mladenovic, "Variable neighborhood search: principles and applications", *European Journal of Operational Research*, 130, pp 449-467, 2001
- [4] Lee, Y. H., M. Pinedo, "Scheduling jobs on parallel machines with sequence-dependent setup times", *European Journal of Operational Research*, 100, pp 464-474, 1997
- [5] Liao, C.-J., H.-C. Juan, "An ant colony optimization for single-machine tardiness scheduling with sequence-dependent setups", *Computers & Operations Research*, pp 1899-1909, 2007
- [6] Lin, S.-W., K.-C. Ying, "Solving single machine total weighted tardiness problems with sequence dependent setup times by meta-heuristics", *International Journal of Advanced Manufacturing Technology*, 34, pp 1183-1190, 2007
- [7] Mladenovic, N., P. Hansen, "Variable Neighborhood Search", *Computers & Operations Research*, 24, pp 1097-1100, 1997
- [8] Mönch, L., R. Driessel "A Distributed Shifting Bottleneck Heuristic for Complex Job Shops. *Computers & Industrial Engineering*", 49, pp 673-680, 2005
- [9] Ovacik, I. M., R. Uzsoy, "Rolling horizon procedures for dynamic parallel machine scheduling with sequence dependent setup times", *International Journal of Production Research*, 33, pp 3173 - 3292, 1995
- [10] Pfund, M., J. W. Fowler, A. Gadkari, Y. Chen, "Scheduling jobs on parallel machines with setup times and ready times", *Computers & Industrial Engineering*, 54, pp 764-782, 2008
- [11] Rocha, M. L., M. G. Ravetti, G. R. Mateus, P. M. Pardalos, "Solving parallel machines scheduling problems with sequence-dependent setup times using variable neighborhood search", *IMA Journal of Management Mathematics*, 18, pp 101-115, 2007
- [12] Wang, X. P., Tang, L. X. "A population-based variable neighborhood search for the single machine total weighted tardiness problem", *Computers & Operations Research*, 36(6), pp 2105-2110, 2009